VisaGuide

Tarea de mejora al proceso 5

Reflexión de aplicación de Lean Software Development

Universidad del Valle de Guatemala · Facultad de Ingeniería

CC3091 · Ingeniería de Software 2 · Semestre II de 2026

Diego Quan (24336) · Diego Guevara (24128)

Juan Francisco Orozco Mijangos (24647) · Norman Aguirre (24479)

Fecha de análisis: 24 de octubre de 2026

Borrador para revisión del grupo. Los porcentajes actuales son estimaciones propuestas; todavía no representan un acuerdo confirmado del equipo.

Contexto y alcance

VisaGuide acompaña a solicitantes durante su proceso de visa estadounidense mediante tres vistas: cliente, asesor y administrador. Incluye selección de perfil, documentos, formulario DS-160, pagos por transferencia, citas y preparación de entrevistas. El proyecto utiliza React, Node.js, Express y PostgreSQL.

Comparamos nuestra aplicación de los siete principios de Lean con la reflexión inicial del semestre. La consigna menciona el 4 de julio; el documento de Tarea 1 proporcionado está fechado el 7 de julio de 2026. Conservamos exactamente sus porcentajes y tomamos ese documento como línea base.

La reflexión combina experiencias comunicadas por el equipo con ejemplos verificables del código, la documentación y el historial de Git. El equipo confirmó que incorporó el rol de asesor al comprender mejor las necesidades y que el feedback del cliente orientó mejoras de UI, modo oscuro y modo senior. Los comentarios sobre RTF y puntualidad también proceden del equipo; no contamos con un registro de frecuencia de reuniones ni con los reportes recientes de Jira.

Criterio de valoración

Los porcentajes expresan una autoevaluación de prácticas y dificultades; no son cobertura de pruebas ni una medición automática. Para la propuesta actual usamos estos rangos orientativos: 50-69 % para aplicación parcial, 70-84 % para prácticas establecidas con brechas importantes y 85-94 % para aplicación consistente con mejoras pendientes. No asignamos 100 % porque siguen existiendo limitaciones de integración, documentación y seguimiento.

Comparación de los siete principios

Principio	Inicial	Actual propuesto	Cambio	Justificación y mejora o retroceso
Eliminar desperdicios	**65 %**	**85 %**	**+20 pp**	Antes había configuración confusa y CSS repetido. Se retiraron archivos y lógica antigua, se modularizó el backend y se separaron responsabilidades del DS-160. Continúan existiendo implementaciones de navegación por rol que requieren mantenimiento.
Amplificar el aprendizaje	**55 %**	**85 %**	**+30 pp**	Antes compartíamos repositorio y retrospectivas, pero faltaban revisiones sistemáticas. Según la experiencia del equipo, las RTF y la comunicación de fallos mejoraron el aprendizaje; ahora existen además ejemplos de pruebas, documentación técnica y correcciones trazables.
Tomar decisiones lo más tarde posible	**70 %**	**80 %**	**+10 pp**	Incorporamos el rol de asesor después de reconocer que admin y cliente no cubrían lo solicitado. Ajustamos la distribución de responsabilidades al comprender mejor el proceso. La abstracción de almacenamiento mantiene opciones abiertas; falta registrar formalmente las decisiones.
Entregar lo antes posible	**85 %**	**90 %**	**+5 pp**	Inicialmente entregábamos por sprint, pero concentrábamos trabajo al final. El equipo reporta mayor puntualidad y responsabilidad; Git muestra cambios incrementales y PR. Falta contrastar la mejora con las métricas recientes de Jira.
Potenciar al equipo	**75 %**	**90 %**	**+15 pp**	Antes predominaba la especialización por integrante. El equipo reporta mayor colaboración para resolver fallos y asumir compromisos. El historial registra aportes de varios integrantes; aún falta demostrar rotación sistemática y documentar acuerdos.
Crear la integridad	**60 %**	**90 %**	**+30 pp**	Antes no había pruebas automatizadas ni CI. Ahora existen pruebas de frontend, backend, integración, autorización y PostgreSQL real, además de CI con lint y build. Playwright y k6 complementan la estrategia, aunque no corren en ese pipeline.
Visualizar todo el conjunto	**70 %**	**85 %**	**+15 pp**	Integramos al asesor al proceso, mejoramos UI y accesibilidad con feedback del cliente y añadimos métricas e informes Excel para admin. Las pruebas entre roles y E2E amplían la visión del producto; aún debemos fortalecer la integración completa.

Promedio inicial: 68,57 %. Promedio actual propuesto: 86,43 %. Mejora: 17,86 puntos porcentuales.

El promedio asigna el mismo peso a cada principio y sirve como síntesis de la autoevaluación. Un aumento de 65 % a 85 % equivale a 20 puntos porcentuales; no significa que el tiempo de desarrollo se haya reducido 20 %.

Reflexión: desperdicios y aprendizaje

1. Eliminar desperdicios

Desperdicios identificados y reducción. Durante el desarrollo encontramos errores en el flujo entre las vistas de cliente, asesor y administrador, archivos antiguos que cubrían responsabilidades ya atendidas y lógica demasiado concentrada. Esto provocaba retrabajo: corregir una pantalla no garantizaba que el proceso completo funcionara.

Un ejemplo concreto es el retiro de `frontend/src/Upload.jsx` el 16 de octubre: la carga vigente se realiza desde la pantalla de documentos y sus componentes. También se eliminó `visa-app/temp_app.js` y lógica antigua sin uso. Estas acciones redujeron fuentes de confusión sobre qué implementación debíamos mantener.

Otro avance fue separar el backend antes concentrado en `app.js` en rutas, controladores y servicios. En el frontend, el DS-160 se dividió en campos reutilizables, definición de secciones, un hook y validaciones. El beneficio principal es localizar mejor los cambios y evitar repetir soluciones; no afirmamos una mejora de rendimiento que no haya sido medida.

Prácticas más efectivas. Nos ayudaron las revisiones técnicas, comunicar fallos al resto del grupo y agregar pruebas de regresión para los contratos entre módulos. La auditoría administrativa, por ejemplo, documentó que una pantalla enviaba `status` mientras el controlador esperaba `estado`. El controlador actual acepta ambos nombres y las pruebas de revisión documental ejercitan las dos variantes.

Requisitos poco claros y cambios innecesarios. La documentación de endpoints en OpenAPI y la definición compartida del flujo ayudan a aclarar qué debe enviar y recibir cada vista. Para continuar mejorando debemos acordar los criterios de aceptación antes de iniciar una tarea y verificar el efecto del cambio en los tres roles, evitando resolver cada pantalla por separado.

2. Amplificar el aprendizaje

Métodos y frecuencia. Según nuestra experiencia, aprendimos mediante trabajo en equipo, RTF, revisión de cambios y comunicación de errores encontrados durante el desarrollo. El repositorio conserva PR, documentación y pruebas que permiten estudiar las soluciones. No tenemos un registro suficiente para afirmar que hubo programación en pareja formal o reuniones con una frecuencia fija; esos métodos no se presentan como prácticas ya establecidas.

Cambio respecto del inicio. En julio, compartir el repositorio era una de nuestras principales formas de aprendizaje. Actualmente podemos revisar un fallo, identificar su causa y conservar una prueba que explique el comportamiento esperado. Compartir conocimiento dejó de depender solamente de comentar una solución: también queda reflejado en código reutilizable y documentación.

Habilidades desarrolladas. Mejoramos la coordinación, el diagnóstico de errores de integración, la separación de responsabilidades y la comprensión de autenticación y permisos. Las pruebas de documentos muestran cómo una aprobación administrativa debe verse en el cliente; las de autorización del asesor recuerdan que un usuario solo debe acceder a los trámites que tiene asignados. Estos casos ayudan a entender partes del proyecto más allá de la pantalla que implementó cada integrante.

Reflexión: decisiones y entregas

3. Tomar decisiones lo más tarde posible

Decisiones que se beneficiaron de obtener más información. Comenzamos con los roles de administrador y usuario. Al comprender mejor lo solicitado, identificamos que esas dos vistas no cubrían las responsabilidades de acompañamiento y revisión del trámite. Decidimos incorporar al asesor en lugar de mantener como definitiva la distribución inicial. Esta evolución permitió distinguir las tareas operativas del asesor de la gestión administrativa y la participación del solicitante.

El aprendizaje fue ajustar una decisión con información del proceso real, sin mantener rígidamente el primer diseño. Como antecedente, la Tarea 1 documenta que evaluamos el almacenamiento antes de elegir Cloudflare R2. Actualmente `backend/storage.js` conserva una abstracción entre el proveedor y los módulos consumidores, facilitando pruebas y cambios de entorno.

Dificultades y mejora pendiente. Ha sido difícil aplicar el principio cuando una decisión afecta simultáneamente al cliente, asesor y administrador. Si cada vista interpreta de manera distinta el estado de un documento o una etapa del trámite, decidir sin acordar el contrato genera retrabajo. Debemos distinguir entre esperar información útil y atrasar acuerdos necesarios: los contratos compartidos deben definirse a tiempo para permitir la integración.

Proponemos registrar las decisiones importantes con el problema, las alternativas, el criterio de elección y la fecha límite para decidir. El 80 % propuesto reconoce la incorporación del asesor a partir de una mejor comprensión del proceso, pero conserva una brecha por la falta de registros formales de decisiones.

4. Entregar lo antes posible

Tiempo entre entregas. Al principio realizábamos menos avances durante el sprint y dejábamos gran parte del trabajo para los últimos días. Esto afectaba la gráfica de Jira y reducía el tiempo para integrar, revisar y corregir. La Tarea 1 ya reconocía esa concentración final, aunque asignaba 85 % por entregar incrementos funcionales en cada sprint.

Según la experiencia del grupo, después aprendimos a entregar las tareas a tiempo, asumir compromisos con más responsabilidad y comunicar bloqueos. El historial actual muestra cambios separados para funcionalidades, correcciones y pruebas, en lugar de una sola modificación final. Esto facilita revisar y detectar problemas antes del cierre.

La mejora se considera cualitativa: sin exportaciones de Jira no podemos calcular cuánto disminuyó el tiempo de ciclo ni afirmar que el burndown ya tiene una forma determinada. Tampoco equiparamos más commits con mayor valor; una entrega útil debe estar terminada, integrada y responder a una necesidad del usuario.

Retroalimentación que cambió la entrega. El cliente nos dio feedback que utilizamos para mejorar la interfaz, el modo oscuro y el modo senior. Esto llevó a priorizar que las funciones fueran claras y utilizables, incluyendo tamaño de elementos y legibilidad. También añadimos métricas e informes exportables a Excel para que el administrador pudiera consultar y aprovechar la información del sistema. Estas mejoras ampliaron el valor de las entregas más allá de completar pantallas.

La retroalimentación técnica también influyó: el E2E detectó que abrir el simulador podía cerrar la sesión por una solicitud sin token. Corregir ese recorrido se volvió necesario para entregar una función utilizable.

Como siguiente paso, proponemos revisar avances a mitad del sprint y reservar tiempo para integración y correcciones, en vez de considerar finalizada una tarea solo porque su código ya fue subido.

Reflexión: equipo e integridad

5. Potenciar al equipo

Participación, confianza y autonomía. Inicialmente cada integrante elegía tareas relacionadas con sus fortalezas, pero eso podía concentrar el conocimiento en una sola persona por área. Según nuestra experiencia, las revisiones técnicas y la comunicación de fallos nos ayudaron a trabajar mejor en conjunto y a buscar soluciones compartidas. También aprendimos a asumir las fechas de entrega como compromisos del equipo.

El historial registra aportes de distintos integrantes en refactorización del backend, DS-160, navegación y limpieza de código. Esa participación respalda la colaboración técnica; por sí sola no demuestra que todas las decisiones hayan sido consensuadas ni que los roles hayan rotado formalmente.

Autonomía y objetivos comunes. El principal desafío ha sido permitir que cada persona avance sin que las decisiones locales rompan la integración. Los errores entre vistas mostraron que terminar una pantalla no equivale a completar el proceso. La autonomía necesita contratos comunes, comunicación de bloqueos y validación del impacto en los otros roles.

Proponemos conservar responsables por tarea y sumar revisión cruzada en cambios que afectan a más de una vista. También debemos compartir decisiones técnicas y documentar acuerdos para reducir la dependencia de quien conoce mejor un módulo. El 90 % propuesto reconoce el aprendizaje relatado, manteniendo la necesidad de confirmar estas prácticas de manera más sistemática.

6. Crear la integridad

Mejoras en pruebas e integración continua. En julio reportábamos Docker y validaciones del DS-160, pero no había pruebas automatizadas ni pipeline. Actualmente el backend utiliza Jest y Supertest; el frontend, Vitest y Testing Library. Hay pruebas de autenticación, permisos, documentos, DS-160, pagos, citas y componentes de las vistas.

Además, existen pruebas HTTP y SQL contra PostgreSQL 15 real. Son relevantes porque los mocks pueden aceptar una consulta o respuesta simulada que no funcionaría contra la base real. Playwright recorre el proceso del cliente en navegador, con acciones del personal realizadas por API; no equivale a probar por interfaz todas las pantallas administrativas.

GitHub Actions está configurado para ejecutar pruebas del backend y PostgreSQL real, pruebas del frontend, ESLint y build en push y PR hacia `main`. También ejecuta auditorías de dependencias y revisión de nombres de archivos sensibles. Esto constituye integración continua; no afirmamos que exista despliegue continuo automático. Playwright y k6 están disponibles por separado y no se ejecutan en ese pipeline.

Dificultades y respuesta. Un ejemplo documentado por el E2E fue la llamada a preguntas del simulador sin token, que recibía 401 y provocaba el cierre de sesión. La documentación registra la corrección y existe una prueba de regresión. La incompatibilidad `status`/`estado` en documentos mostró otra limitación: una prueba aislada puede pasar y aun así representar mal el contrato real.

Para continuar debemos ampliar los recorridos entre roles, mantener pruebas con la base real y considerar incorporar los E2E al CI con recursos adecuados. El 90 % propuesto reconoce la mejora técnica sin asumir cobertura total ni ausencia de errores. En este análisis inspeccionamos código y configuración; no ejecutamos nuevamente las suites ni certificamos el resultado actual del pipeline.

Reflexión: visualizar todo el conjunto

7. Del módulo individual al proceso completo

En la primera reflexión ya contábamos con Design Thinking, User Story Mapping y diagramas de arquitectura. Después reconocimos que el modelo de administrador y usuario necesitaba un asesor para completar lo solicitado. Incorporar ese rol nos obligó a pensar en asignaciones, permisos, revisión y comunicación como partes de un mismo proceso. Los errores posteriores entre vistas mostraron que cada cambio debe producir un resultado coherente en las demás.

La revisión documental es un ejemplo concreto. El cliente carga un archivo, el personal lo revisa y el cliente debe ver la aprobación o la corrección con sus observaciones. La suite `documentsReview.integration.test.js` verifica ese intercambio, incluida la sustitución de un documento corregido. Las pruebas de autorización del asesor cubren además la restricción a los trámites asignados.

Otro avance es disponer de un servicio del flujo del cliente, un hook que consulta su estado y protección de pasos en el frontend y backend. El objetivo es que las pantallas se basen en reglas compartidas y que ocultar una opción visual no sea el único control. Estos elementos requieren mantenerse alineados cuando cambian los requisitos.

También ampliamos la visión del producto al mejorar la UI, el modo oscuro y el modo senior con feedback del cliente. Las métricas e informes Excel del administrador complementan el seguimiento operativo del trámite. El repositorio incluye la descarga desde `AdminReports.jsx` y la generación del archivo mediante `adminExcelReport.js`. Así consideramos tanto el uso por el solicitante como las necesidades de supervisión del personal.

Sidebar: alcance real de la unificación

El commit `2bedfbd` del 19 de octubre se titula "unificar comportamiento de sidebars" y modifica los tres roles. Simplifica estados y controles de expansión para armonizar su comportamiento. Sin embargo, el cliente conserva `Sidebar.jsx`, el administrador `AdminLayout.jsx` y el asesor `AdvisorLayout.jsx`.

Por eso describimos el avance como estandarización del comportamiento, con reutilización parcial de marca, tema y estilos. No afirmamos que exista un único componente de sidebar para toda la aplicación. Una mejora futura sería evaluar una estructura común configurable por rol, siempre que reduzca duplicación sin ocultar diferencias necesarias.

Ejemplos adicionales de reutilización

- `documentPreview.js` concentra la apertura de documentos y es utilizado en las vistas de administrador y asesor.

- `apiClient.js` concentra URL, tiempos de espera y manejo de errores; las utilidades de solicitudes administrativas y del asesor lo reutilizan.

- `useDS160Form.js`, `ds160Validation.js`, `DS160Field.jsx` y `ds160Sections.js` separan estado, validaciones, campos y configuración del formulario.

- El backend organiza rutas, controladores y servicios para localizar reglas de negocio y facilitar pruebas.

- `lazyRoutes.js` carga páginas de manera diferida y permite precargar rutas. Es una estrategia de carga; no atribuimos una reducción concreta de tiempo sin una medición comparativa.

Estas prácticas respaldan el 85 % actual propuesto: pensamos más en interacciones y mantenimiento, pero siguen pendientes validaciones completas entre roles y métricas de uso real. También debemos actualizar la documentación cuando cambia un contrato para que no convivan explicaciones históricas con reglas vigentes sin distinguirlas.

Conclusiones y próximos compromisos

Nuestra aplicación de Lean pasó de una base centrada en organización, prototipos y entregas por sprint a un proceso con más atención a la calidad, la integración y la responsabilidad compartida. El avance técnico más claro está en crear integridad: de no contar con pruebas automatizadas ni CI, pasamos a disponer de pruebas en varias capas y validaciones automáticas en GitHub Actions.

La eliminación de desperdicios también tiene resultados concretos. Retiramos implementaciones antiguas y separamos responsabilidades en el backend y el DS-160. El aprendizaje es que agregar archivos puede ser una mejora cuando cada uno concentra una responsabilidad y evita repetir lógica; eliminar archivos aporta valor cuando retira fuentes redundantes o abandonadas.

En el trabajo del grupo, aprendimos que dejar las tareas para el final afectaba la integración y la capacidad de corregir. Según nuestra experiencia posterior, mejoramos la puntualidad y la responsabilidad. Debemos sostener ese cambio con entregas útiles y revisables durante el sprint, y respaldarlo con métricas de Jira. La cantidad de commits es una señal de actividad, pero no reemplaza la evaluación de tareas terminadas.

También aprendimos a ajustar el producto al comprender mejor las necesidades: incorporamos al asesor porque admin y usuario no completaban lo solicitado, mejoramos la UI y los modos oscuro y senior con feedback del cliente, y añadimos métricas e informes Excel para administración. Los fallos entre roles mostraron que estas mejoras deben validarse como parte del proceso completo. Las RTF, la comunicación de errores y las pruebas de integración ayudan a convertir esos problemas en aprendizaje y prevención.

Con la valoración propuesta, el promedio general cambia de **68,57 % a 86,43 %**, una mejora de **17,86 puntos porcentuales**. Los mayores aumentos corresponden a amplificar el aprendizaje y crear integridad, con 30 puntos cada uno. Entregar lo antes posible aumenta cinco puntos porque la línea base ya era alta y su mejora debe contrastarse con los reportes recientes. Ningún principio recibe 100 % y no proponemos retrocesos generales, aunque sí reconocemos fallos de integración durante el camino.

Acciones para sostener la mejora

1. Revisar los porcentajes actuales como grupo y ajustar cualquier valor que no represente la experiencia compartida.

2. Acordar contratos y criterios de aceptación entre roles antes de implementar cambios que afectan el flujo.

3. Registrar decisiones importantes, alternativas consideradas y razones de elección.

4. Dar seguimiento durante el sprint a tareas terminadas, bloqueos y tiempo de ciclo con Jira.

5. Mantener pruebas de regresión y ampliar los recorridos entre cliente, asesor y administrador; evaluar E2E en CI.

Fuentes del análisis

Documentos: consigna de Tarea 5 y reflexión de Tarea 1 proporcionadas por el equipo. Experiencias del grupo: RTF, colaboración, errores, evolución de entregas, incorporación del asesor y feedback del cliente sobre UI y modos visuales.

Repositorio local: auditoría administrativa; documentación E2E; pruebas de revisión documental, autorización y PostgreSQL; `.github/workflows/ci.yml`; componentes y utilidades mencionados. Historial: `c8487d5` (backend), `fb0c23b` (DS-160), `0a48dd3` y `782fb35` (limpieza), `2bedfbd` (sidebars), `4fd1c7b` (panel asesor) y `b443163` (Excel).

Antes de entregar: revisar los porcentajes propuestos como grupo. Si se dispone del dato, agregar la frecuencia de las RTF y reuniones. La valoración actual fue propuesta a solicitud del equipo y sigue abierta a ajustes; no se asignó una frecuencia de revisión sin confirmación.